

Decision Trees and Random Forests

Please reference the video lecture for an overview of Decision Trees and Random Forests. This is just a reference notebook for the lecture video's code.

Growing a Decision Tree

You may need to install the rpart library. you can find a lot more information about this library [here](#).

In [2]:

```
#install.packages('rpart')
```

In [4]:

```
library(rpart)
```

We can then use the rpart() function to build a decision tree model:

rpart(formula, data=, method=,control=) where

- the formula is in the format: outcome ~ predictor1+predictor2+predictor3+ect.
- data= specifies the data frame
- method= "class" for a classification tree
- "anova" for a regression tree
- control= optional parameters for controlling tree growth.
 - For example, control=rpart.control(minsplit=30, cp=0.001) requires that the minimum number of observations in a node be 30 before attempting a split and that a split must decrease the overall lack of fit by a factor of 0.001 (cost complexity factor) before being attempted.

Sample Data

We'll use the [kyphosis](#) data frame which has 81 rows and 4 columns. representing data on children who have had corrective spinal surgery. It has the following columns:

- Kyphosis-a factor with levels absent present indicating if a kyphosis (a type of deformation) was present after the operation.
- Age-in months
- Number-the number of vertebrae involved
- Start-the number of the first (topmost) vertebra operated on.

Let's check out the structure:

In [34]:

```
str(kyphosis)
```

```
'data.frame': 81 obs. of 4 variables:
 $ Kyphosis: Factor w/ 2 levels "absent","present": 1 1 2 1 1 1 1 1 1 2 ...
 $ Age      : int  71 158 128 2 1 1 61 37 113 59 ...
 $ Number   : int  3 3 4 5 4 2 2 3 2 6 ...
 $ Start    : int  5 14 5 1 15 16 17 16 16 12 ...
```

In [6]:

```
head(kyphosis)
```

Out[6]:

```
  Kyphosis Age Number Start
```

	Kyphosis	Age	Number	Start
1	absent	71	3	5
2	absent	158	3	14
3	present	128	4	5
4	absent	2	5	1
5	absent	1	4	15
6	absent	1	2	16

In [8]:

```
tree <- rpart(Kyphosis ~ . , method='class', data= kyphosis)
```

Examining Results of the Tree Model

There are lots of functions you can use to examine your tree model:

</table></p>

printcp(fit)	display cp table
plotcp(fit)	plot cross-validation results
rsq.rpart(fit)	plot approximate R-squared and relative error for different splits (2 plots). labels are only appropriate for the "anova" method.
print(fit)	print results
summary(fit)	detailed results including surrogate splits
plot(fit)	plot decision tree
text(fit)	label the decision tree plot
post(fit, file=)	create postscript plot of decision tree

Let's see a few of them:

In [29]:

```
printcp(tree)
```

Classification tree:
rpart(formula = Kyphosis ~ ., data = kyphosis, method = "class")

Variables actually used in tree construction:
[1] Age Start

Root node error: 17/81 = 0.20988

n= 81

	CP	nsplit	rel error	xerror	xstd
1	0.176471	0	1.00000	1.0000	0.21559
2	0.019608	1	0.82353	1.0588	0.22010
3	0.010000	4	0.76471	1.0588	0.22010

Tree Visualization

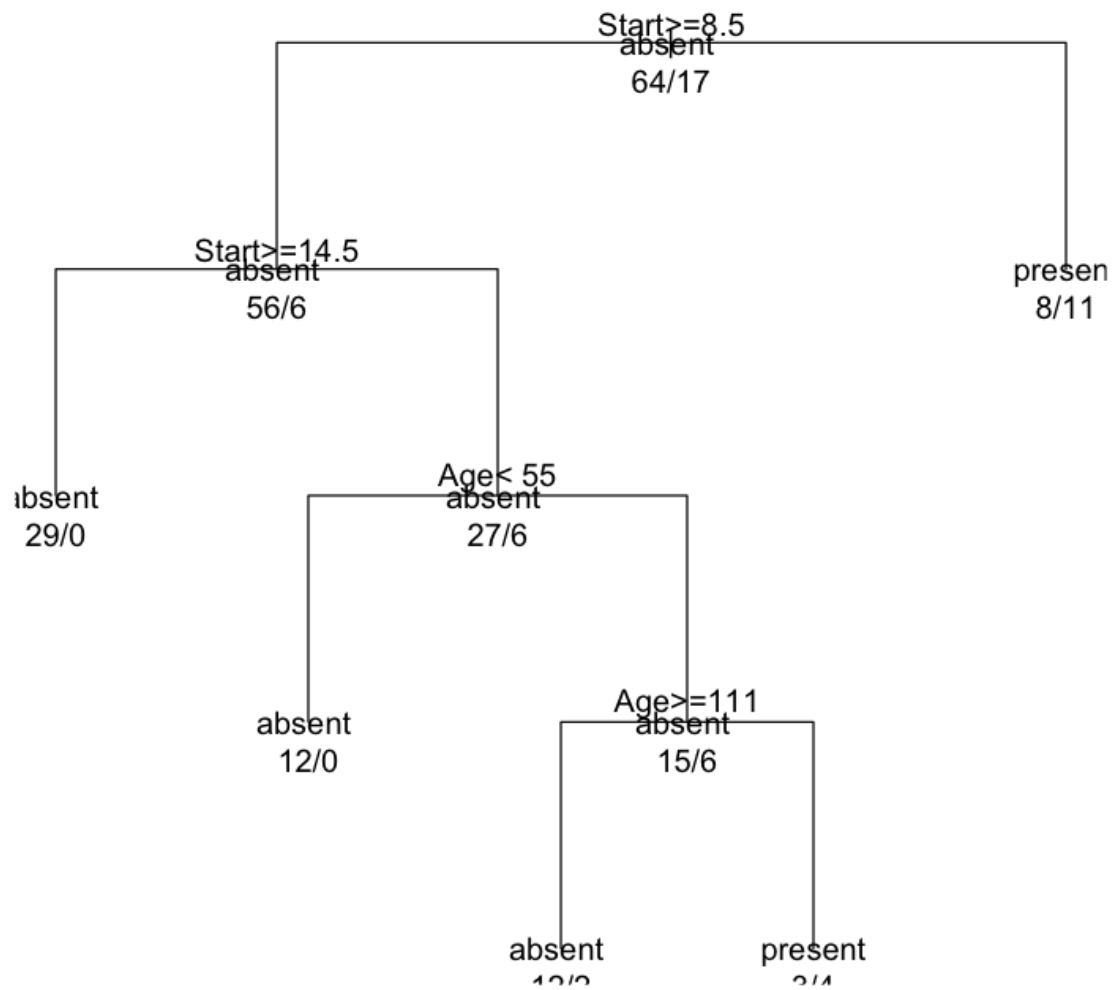
There are some built-in visualization capabilities from the table above, but they aren't very good looking:

In [35]:

```
plot(tree, uniform=TRUE, main="Main Title")
```

```
plot(tree, main=" ", use.n=TRUE, all=TRUE)
text(tree, use.n=TRUE, all=TRUE)
```

Main Title



The `rpart.plot` library package makes these visualizations much better.

In [36]:

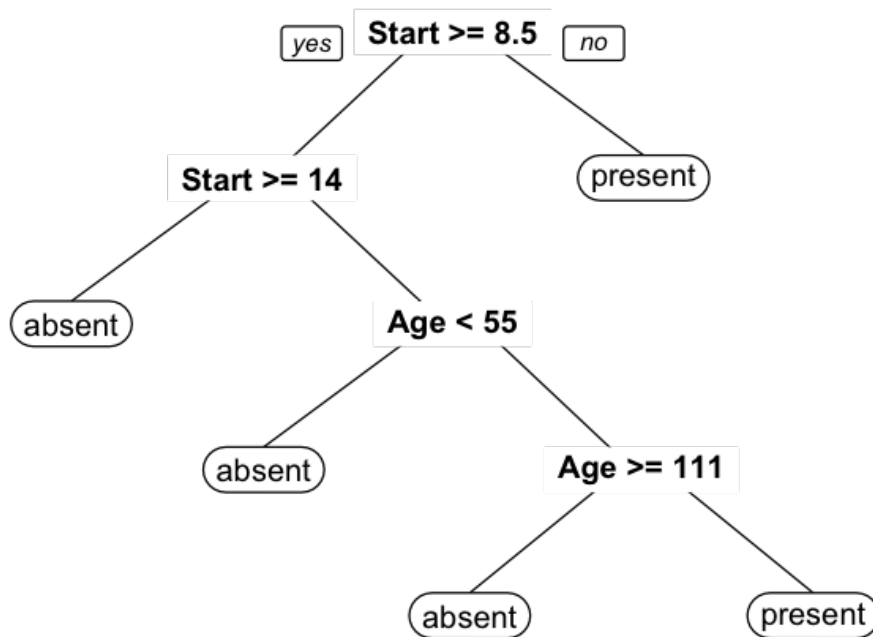
```
#install.packages('rpart.plot')
```

In [37]:

```
library(rpart.plot)
```

In [33]:

```
prp(tree)
```



Random Forests

Random forests improve predictive accuracy by generating a large number of bootstrapped trees (based on random samples of variables), classifying a case using each tree in this new "forest", and deciding a final predicted outcome by combining the results across all of the trees (an average in regression, a majority vote in classification).

We can use the `randomForest` library to create and build out a Random Forest:

In [39]:

```
# Random Forest prediction of Kyphosis data  
library(randomForest)
```

In [40]:

```
model <- randomForest(Kyphosis ~ ., data=kyphosis)
```

In [41]:

```
print(model) # view results
```

Call:

```
randomForest(formula = Kyphosis ~ ., data = kyphosis)
  Type of random forest: classification
    Number of trees: 500
No. of variables tried at each split: 1
```

```
      OOB estimate of  error rate: 19.75%
Confusion matrix:
      absent present class.error
absent     60      4  0.0625000
present    12      5  0.7058824
```

In [20]:

```
importance(model) # importance of each predictor
```

Out[20]:

	MeanDecreaseGini
Age	8.739632
Number	5.497958
Start	9.998735

Conclusion

You should be beginning to feel very comfortable with the syntax for training a model on data. The key is to just understand the background of the algorithm being used and know what library to install and use for the specific algorithm being used.